

torch.nn.utils.stateless.functional_call#

https://pytorch.org/docs/stable/generated/torch.nn.utils.stateless.functional_call.html

Description:

Perform a functional call on the module by replacing the module parameters and buffers with the provided ones. This API is deprecated as of PyTorch 2.0 and will be removed in a future version of PyTorch. Please use `torch.func.functional_call()` instead, which is a drop-in replacement for this API. If the module has active parametrizations, passing a value in the `parameters_and_buffers` argument with the name set to the regular parameter name will completely disable the parametrization. If you want to apply the parametrization function to the value passed please set the key as `{submodule_name}.parametrizations.{parameter_name}.original`.

Function Signature

```
torch.nn.utils.stateless.functional_call(module,  
parameters_and_buffers, args=None, kwargs=None, *,  
tie_weights=True, strict=False)[source]#
```

Parameters

Returns

Return type

Returns:

Any

Code Examples

```
>>> a = {'foo': torch.zeros(())}
>>> mod = Foo() # does self.foo = self.foo + 1
>>> print(mod.foo) # tensor(0.)
>>> functional_call(mod, a, torch.ones(()))
>>> print(mod.foo) # tensor(0.)
>>> print(a['foo']) # tensor(1.)
```

```
>>> a = {'foo': torch.zeros(())}
>>> mod = Foo() # has both self.foo and self.foo_tied which are
# tied. Returns x + self.foo + self.foo_tied
>>> print(mod.foo) # tensor(1.)
>>> mod(torch.zeros(())) # tensor(2.)
>>> functional_call(mod, a, torch.zeros(())) # tensor(0.) since
# it will change self.foo_tied too
>>> functional_call(mod, a, torch.zeros()), tie_weights=False)
# tensor(1.)--self.foo_tied is not updated
>>> new_a = {'foo': torch.zeros(()), 'foo_tied':
# torch.zeros()}
>>> functional_call(mod, new_a, torch.zeros()) # tensor(0.)
```

Notes & Warnings

 **WARNING** This API is deprecated as of PyTorch 2.0 and will be removed in a future version of PyTorch. Please use `torch.func.functional_call()` instead, which is a drop-in replacement for this API.

NOTE

Note If the module has active parametrizations, passing a value in the `parameters_and_buffers` argument with the name set to the regular parameter name will completely disable the parametrization. If you want to apply the parametrization function to the value passed please set the key as `{submodule_name}.parametrizations.{parameter_name}.original`.

NOTE

Note If the module performs in-place operations on parameters/buffers, these will be reflected in the `parameters_and_buffers` input. Example:

```
>>> a = {'foo': torch.zeros()} >>> mod = Foo() # does self.foo = self.foo + 1
>>> print(mod.foo) # tensor(0.) >>> functional_call(mod, a, torch.ones())
>>> print(mod.foo) # tensor(0.) >>> print(a['foo']) # tensor(1.)
```

NOTE

Note If the module has tied weights, whether or not `functional_call` respects the tying is determined by the `tie_weights` flag. Example:

```
>>> a = {'foo': torch.zeros()} >>> mod = Foo() # has both self.foo and self.foo_tied
which are tied. Returns x + self.foo + self.foo_tied >>> print(mod.foo) #
tensor(1.) >>> mod(torch.zeros()) # tensor(2.) >>> functional_call(mod, a,
torch.zeros()) # tensor(0.) since it will change self.foo_tied too >>>
functional_call(mod, a, torch.zeros(), tie_weights=False) # tensor(1.)--
self.foo_tied is not updated >>> new_a = {'foo': torch.zeros(), 'foo_tied':
torch.zeros()} >>> functional_call(mod, new_a, torch.zeros()) # tensor(0.)
```

Detailed Documentation

Documentation

Perform a functional call on the module by replacing the module parameters and buffers with the provided ones.

This API is deprecated as of PyTorch 2.0 and will be removed in a future version of PyTorch. Please use `torch.func.functional_call()` instead, which is a drop-in replacement for this API.

If the module has active parametrizations, passing a value in the `parameters_and_buffers` argument with the name set to the regular parameter name will completely disable the parametrization. If you want to apply the parametrization function to the value passed please set the key as `{submodule_name}.parametrizations.{parameter_name}.original`.

If the module performs in-place operations on parameters/buffers, these will be reflected in the `parameters_and_buffers` input.

If the module has tied weights, whether or not `functional_call` respects the tying is determined by the `tie_weights` flag.

`module` (`torch.nn.Module`) – the module to call

`parameters_and_buffers` (dict of str and Tensor) – the parameters that will be used in the module call.

`args` (Any or tuple) – arguments to be passed to the module call. If not a tuple, considered a single argument.

`kwargs` (dict) – keyword arguments to be passed to the module call

`tie_weights` (bool, optional) – If True, then parameters and buffers tied in the original

model will be treated as tied in the reparamaterized version. Therefore, if True and different values are passed for the tied parameters and buffers, it will error. If False, it will not respect the originally tied parameters and buffers unless the values passed for both weights are the same. Default: True.

strict (bool, optional) – If True, then the parameters and buffers passed in must match the parameters and buffers in the original module. Therefore, if True and there are any missing or unexpected keys, it will error. Default: False.

the result of calling module.